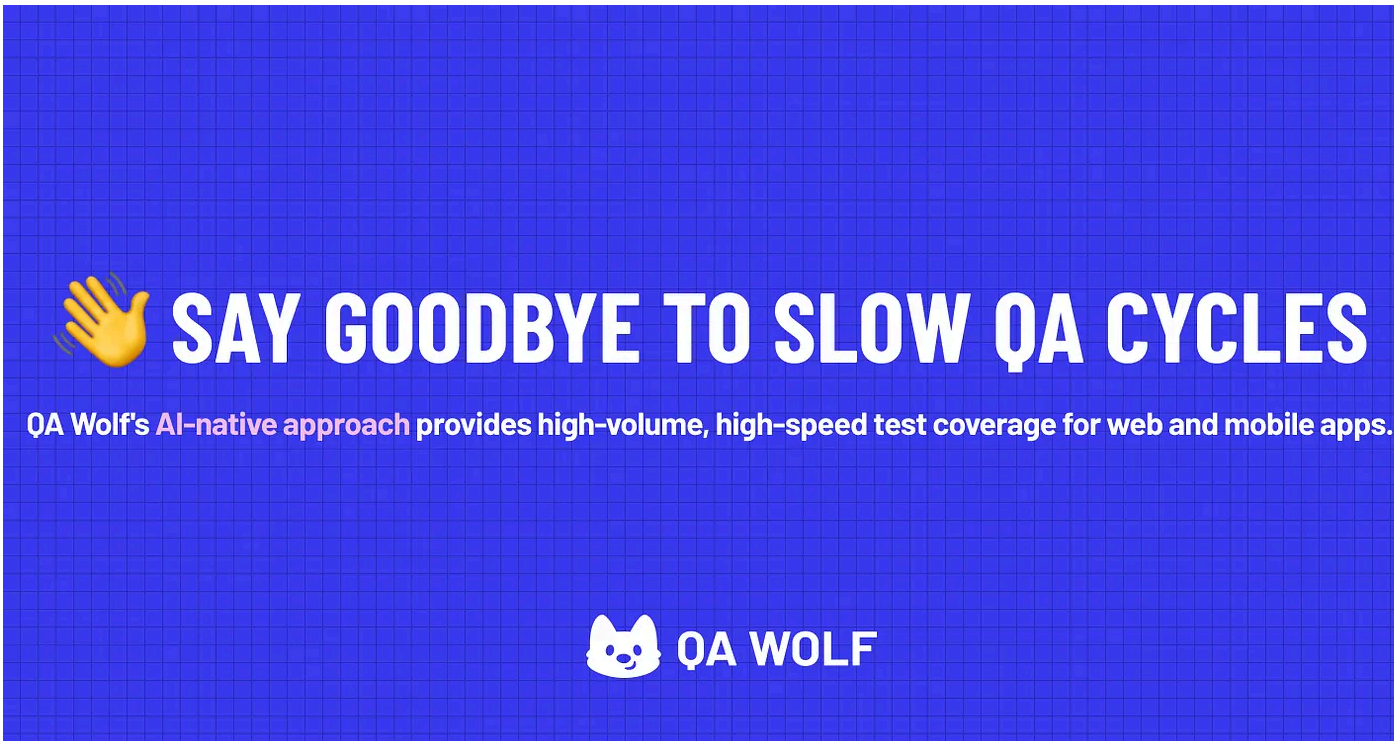
Uber’s Billion Trips Migration Setup with Zero Downtime

BYTEBYTEGO
NOV 19, 2024

Meet your EOY deadlines – faster releases, zero quality compromises (Sponsored)



If slow QA processes and flaky tests are a bottleneck for your engineering team, you need QA Wolf.

Their AI-native platform, backed by full-time QA engineers, enables their team to create tests **5x faster** than anyone else. New tests are created in minutes and existing tests are updated almost instantaneously.

✔ Unlimited parallel test runs
✔ 15-min QA cycles
✔ 24-hour maintenance and on-demand test creation
✔ CI/CD integration
✔ Zero-flake guarantee

🛒 Also available through AWS, GCP, and Azure marketplaces.

Disclaimer: The details in this post have been derived from the Uber Technical Blog. All cre for the technical details goes to the Uber engineering team. The links to the original article and other references are present in the references section at the end of the post. We’ve attempted to analyze the details and provide our input about them. If you find any inaccuracies or omissions, please leave a comment, and we will do our best to fix them.

Maintaining uptime during system upgrades or migrations is essential, especially fo high-stakes, real-time platforms like Uber's trip fulfillment system.

Uber relies on consistent, immediate responsiveness to manage millions of trip requests and transactions daily. The implications of downtime for a platform like U are scary: even a brief service interruption could lead to lost revenue, user dissatisfaction, and huge reputational damage.

However, Uber faced the daunting challenge of migrating its complex fulfillment infrastructure from an on-premises setup to a hybrid cloud environment.

This task not only required a deep understanding of system architecture but also an approach that could guarantee zero downtime, ensuring that millions of riders, drivers, and partners worldwide would experience uninterrupted service during the transition.

In this article, we’ll look at Uber's zero-downtime migration strategy. We’ll also lea more about the technical solutions they implemented and the challenges they faced the process.

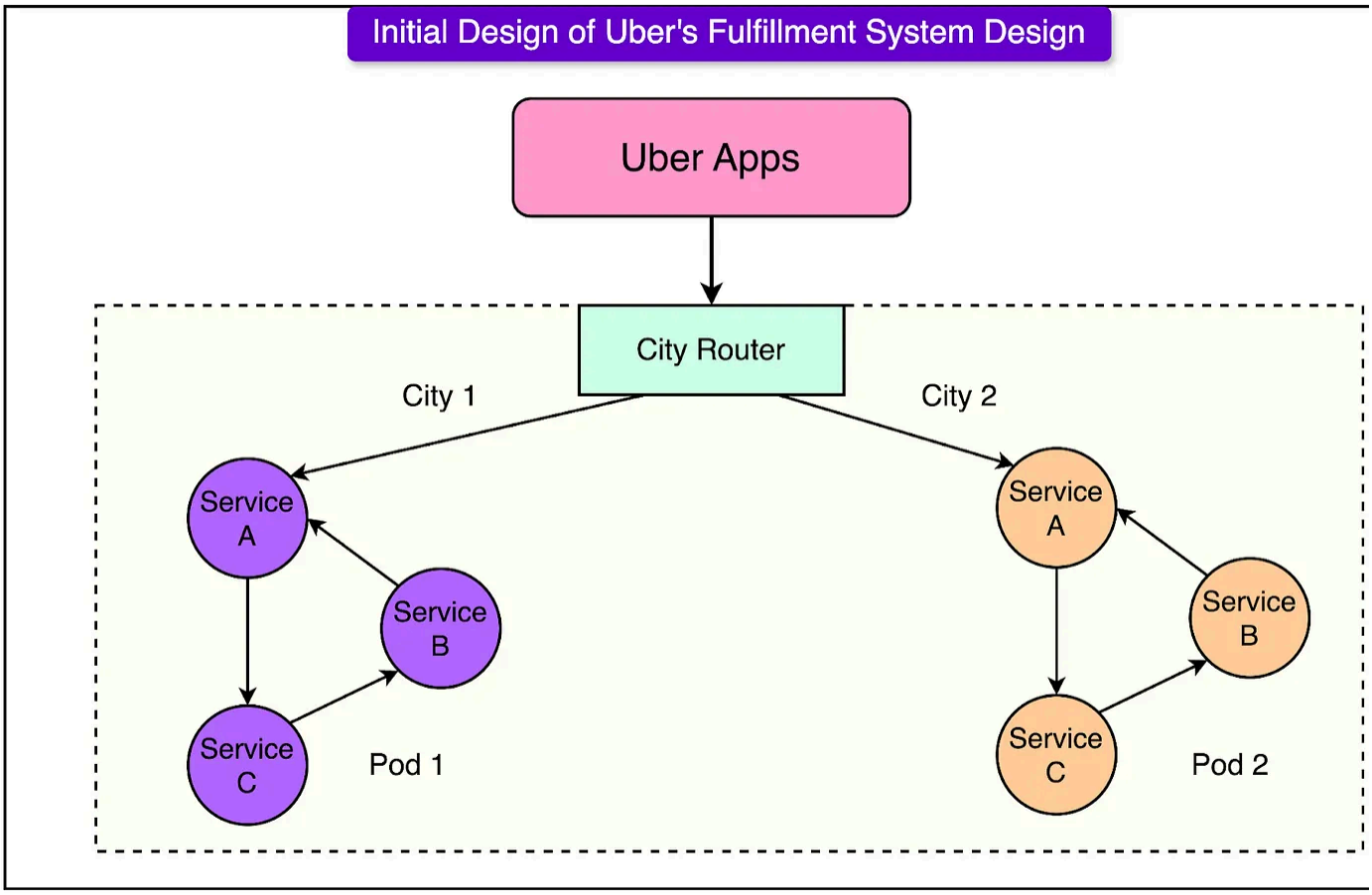
The Complexity of Uber’s Trip Fulfillment Platforr

Uber’s fulfillment system, before it migrated to a hybrid cloud setup, was designed handle vast amounts of real-time data and high transaction volumes.

At its core, the fulfillment system managed the interactions between millions of rid drivers, couriers, and other service elements, processing over two million transactic per second. Some stats to describe the system’s importance are as follows:

- The platform handles more than a million concurrent users and billions of trip per year across over ten thousand cities.
- The platform supports billions of database transactions a day.
- Hundreds of Uber microservices rely on the platform as the source of truth for accurate state of the trips.

The architecture was organized into “pods”, where each pod was a self-contained u of services dedicated to a particular city or geographic region. See the diagram belo



For reference, a pod is a self-sufficient unit with many services interacting with eac other to handle fulfillment for a single city. Once a request enters a pod, it remains within the services in the pod unless it requires access to data from services outside the pod.

Within each pod, services were divided into "demand" and "supply" systems. The demand and supply services were shared-nothing microservices with the entities stored in Apache Cassandra and Redis key-value tables.

- The demand services managed rider interactions
- Supply services handled driver operations.

These services stayed synchronized using distributed transactional techniques to ensure that trip-related data remained consistent within each pod. Uber employed saga pattern for this.

Entity consistency within each service was managed through in-memory data management and serialization.

Architectural Limitations and Scalability Issues

The initial architecture was designed to prioritize availability, sacrificing some aspe of strict data consistency to maintain a robust user experience.

However, as Uber's operations expanded, these architectural choices created bottlenecks:

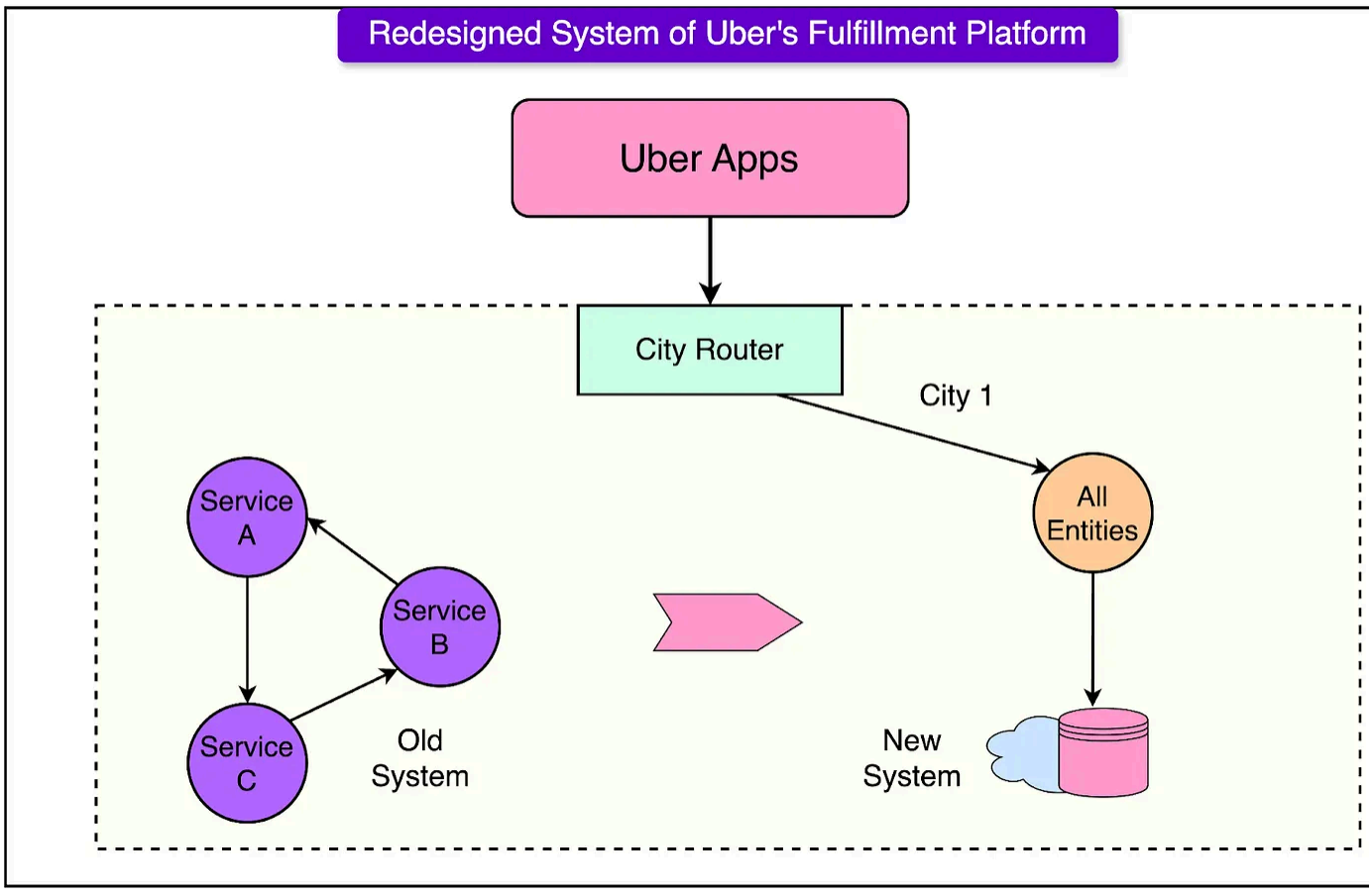
- **Eventual Consistency**: The entire architecture was based on the thought proce of trading off consistency for availability and latency. The lack of atomicity me the need to reconcile when the second operation failed.
- **Multi-Entity Write Operations:** When an operation had to write across multip entities, the application layer handled this interaction using an RPC-based mechanism. The system had to constantly verify the expected and current state fix mismatches.
- **Limited Scalability:** The cities were sharded among one of the available pods a the size of the pod was dependent on the maximum ring size of a cluster. There was a vertical limit for scaling the pod if any of the cities crossed a threshold of concurrent trips.

The Redesigned Architecture

In the redesigned system architecture, the Uber engineering team shifted from a distributed, in-memory setup to a more centralized, cloud-backed infrastructure.

The new system consolidated the previously separate “demand” and “supply” servic into a single application layer supported by a cloud database. By moving data management to a datastore layer, the new system streamlined operations and improved scalability and consistency.

See the diagram below to understand the new architecture:



Key Solutions Implemented By Uber

While designing the new solution had its share of the complexity, the real challenge was migrating the workload to the new design.

Some of the key solutions implemented by Uber’s engineering team to achieve zero downtime migration to the new system are as follows:

1 - Backward Compatibility Layer

Uber implemented a backward compatibility layer as a core component of its zero-downtime migration strategy.

This layer served as a bridge, allowing existing APIs and event contracts to functio normally while Uber transitioned to a new system architecture. By supporting the c API contracts and event schemas, the backward compatibility layer ensured that m internal and external consumers of Uber’s APIs could continue to operate without modification.

See the diagram below to understand the role of the backward compatibility layer.

Some of the benefits of the backward compatibility layer are as follows:

- By keeping old API contracts intact, Uber avoided abrupt changes that could cause errors or service interruptions, ensuring a stable user experience through the migration.
- Consumers could move to the new system at their own pace, enabling a gradua adoption of the new API endpoints without disrupting workflows.
- The compatibility layer minimized the need for coordination across all API consumers, as teams could adopt the new system independently.

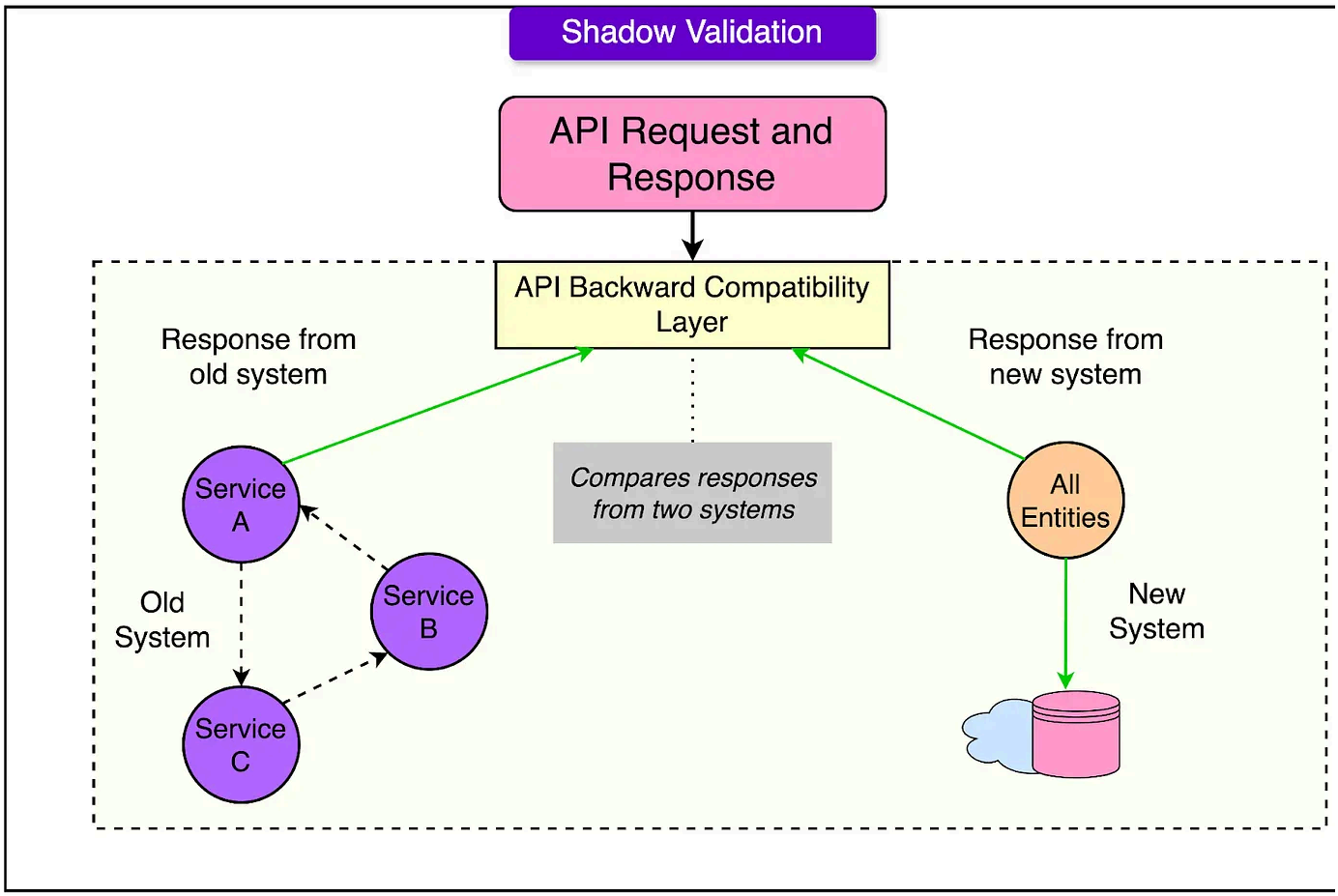
However, there were also some downsides to this:

- **Increased Complexity:** Maintaining a compatibility layer added another source complexity to the migration. Every API endpoint and event schema had to be k consistent across both systems, leading to possible redundancy in data process and storage.
- **Risk of Technical Debt:** Continuing to support outdated APIs and event schem can create long-term technical debt. If not carefully managed, the compatibility layer could lead to ongoing maintenance costs and slow down future developm
- **Performance Overheads:** The compatibility layer could introduce performance overhead, as requests may need to be processed in both the old and new system or converted to fit the new architecture. This can result in latency, especially in high-volume systems.

2 - Shadow Validation

Shadow validation was integrated into Uber’s high-transaction, real-time platform.

Each request sent to the old system was mirrored in the new one, and responses fro both were compared on a key-value basis. Discrepancies were logged and analyzed within Uber’s observability framework, with differences captured in a dedicated observability system for further examination.



This comparison was not simply binary; Uber allowed for certain predefined tolerances and exceptions to accommodate unavoidable variances, such as transien data changes or slight delays due to processing orders.

When differences between systems arose, Uber took a two-fold approach:

- **Automated Logging and Alerts:** Minor discrepancies within pre-approved tolerances were logged for further analysis but did not require immediate actio These were typically edge cases where slight timing differences or transient sta mismatches occurred.
- **Manual Intervention for Critical Mismatches:** If a response deviation posed a risk to user functionality or violated critical data requirements, the engineering team could prioritize it for immediate investigation. This step ensured that hig

impact discrepancies were resolved quickly, reducing the chance of significant issues when the system went live.

Migration Phase

Let's now understand how Uber carried out the migration.

1 - Pre-Rollout Preparations

Uber's pre-rollout preparation steps were foundational to achieving a smooth, zero-downtime migration for their trip fulfillment platform.

Each step, such as shadow validation, end-to-end (E2E) testing, load testing, and database warm-up, played a critical role in minimizing potential risks and ensuring the new system would perform reliably under real-world conditions.

Some of the key activities that were performed are as follows:

End-to-End (E2E) Testing

E2E testing enabled Uber to verify the complete functionality of the new system fro start to finish. It was essential to identify potential integration issues or bottlenecks

By simulating realistic user journeys, these tests checked that all workflows, integrations, and dependencies performed as expected under different scenarios.

Load Testing

Load testing subjected the new system to simulated traffic levels that matched or exceeded actual usage, evaluating its capacity to handle high transaction volumes without degradation.

Load testing confirmed that the new system could withstand Uber's high operation demands, from peak traffic to unexpected surges. It allowed Uber to preemptively

address any performance issues, such as latency or system overload, under stress conditions.

Database Warm-Up

Database warm-up involved generating synthetic data loads to pre-fill caches and s partitions, ensuring the database was primed for full production traffic from the sta

For Uber, whose cloud database had to handle rapid scaling, database warm-up prevented "cold-start" issues by ensuring that common queries and data partitions were already optimized for performance. This step reduced the chance of initial slowdowns or resource bottlenecks during migration.

2 - Traffic Pinning and Phased Rollouts

Uber employed a traffic pinning and phased rollout strategy to migrate specific trip incrementally to the new system to reduce the risk of inconsistencies

This approach allowed Uber to gradually shift parts of its trip fulfillment platform t the new architecture.

Technical Process of Traffic Pinning and Phased Rollout

Traffic pinning ensured that each trip's data was processed by a single system—eitl the old or the new—throughout its lifecycle.

This was critical for preventing data fragmentation and ensuring consistent trip updates, as each trip involves multiple interactions, such as driver updates, route changes, and fare calculations.

To achieve this, Uber developed a routing logic to "pin" ongoing trips to the system where they were initiated. Before migration, consumer identifiers for riders and drivers were recorded, enabling Uber to route each interaction related to a given tri

back to its origin system, preventing mid-trip transitions that could lead to data mismatches.

This tracking persisted until the trip was completed, after which riders and drivers were gradually transitioned to the new system for future trips.

The diagram below demonstrates the concept of traffic pinning.

Source: Uber’s zero-downtime migration

Initially, Uber migrated less critical or idle riders and drivers to the new system, followed by active trips in specific cities. Over time, this phased approach allowed Uber to monitor and control the migration, expanding it to cover larger segments a eventually all active users.

The key benefits of traffic pinning are as follows:

- **Data Consistency**: Traffic pinning ensured that all interactions for a specific tr remained in the same system, preventing data inconsistencies. This consistenc essential for real-time platforms where split-second updates are crucial to the u experience.
- **Reduced Risk of Errors**: By phasing out migrations, Uber could control and monitor each stage, identifying potential issues in a manageable scope. If an er surfaced, Uber could isolate it to a specific set of users or regions, minimizing overall impact.
- **Increased Stability for Users**: Due to this approach, trips in progress weren't affected by system changes, reducing the likelihood of transaction failures or incomplete updates that could disrupt users.

Key Observability and Rollback Mechanisms

Uber developed detailed dashboards and monitoring tools to track metrics like trip volume, trip completion rates, driver availability, and overall system load.

These dashboards provided visibility into performance and data consistency across both systems, allowing engineers to observe how traffic gradually drained from the system while increasing in the new one.

The goal was that the overall aggregate metrics should remain flat. Key metrics, including transaction success rates, latency, and error rates, were monitored at city level granularity to catch any localized disruptions.

These observability tools also enabled Uber to spot irregularities in real-time.

For example, if completion rates dropped in a specific city, engineers could quickly investigate and determine whether the issue originated from the new system. The tools flagged critical problems that could trigger a rollback for specific regions, allowing Uber to maintain stability while continuing the migration elsewhere.

The rollback mechanism was equally important.

Uber could reverse traffic flow for any region back to the old system if metrics indicated significant deviations from expected performance.

Conclusion

Uber’s zero-downtime migration approach highlights key technical strategies for complex, large-scale migrations.

- By implementing a backward compatibility layer, Uber maintained service continuity, allowing gradual, flexible transitions for its consumers.
- The shadow validation and traffic pinning techniques ensured data consistency and stability, while observability and rollback mechanisms provided real-time insights and controlled reversibility in case of issues. These strategies allowed Uber to minimize user impact while migrating critical components.

Despite its success, Uber’s approach faced challenges, such as the high infrastructu demand of maintaining dual systems and the complexity of managing large-scale observability.

References:

- Uber’s Fulfillment Platform: Ground-up Rearchitecture to Accelerate Uber’s Go/Get Strategy
- Building Uber’s Fulfillment Platform for Planet-Scale Using Spanner
- Uber's Blueprint for Zero-Downtime Migration of Complex Trip Fulfillment Platform

SPONSOR US

241 Likes · 8 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Egor Voronianskii Egor's Substack Nov 20, 2024 *Edited*

Great!

LIKE REPLY